Task 3: Adversity Prediction Model

1. Choice of Model and Rationale

Problem framing. The goal is to predict $p(\text{adverse} \mid \text{trade features}, \tau) \in [0, 1]$, i.e. the probability that a given trade will be adverse at horizon τ. This is a binary classification problem with a natural probabilistic output requirement, since downstream tasks (Task 4 threshold sweep, Task 5 quoting) consume the score as a continuous adversity signal rather than a hard label.

Stacked-τ design. Rather than training six separate models (one per horizon), a *single* global model is trained on all horizons simultaneously, with τ included as an explicit input feature. This has two advantages: (i) the model shares statistical strength across horizons, which is especially valuable for longer-horizon labels where signal is weaker; and (ii) it naturally captures the monotone adversity-vs-τ relationship observed in Task 1 without requiring separate calibration runs.

Algorithm: `HistGradientBoostingClassifier`. A histogram-based gradient boosted tree ensemble (scikit-learn `HistGradientBoostingClassifier`) is chosen for the following reasons:

- **Tabular efficiency.** Gradient-boosted trees consistently outperform linear models on structured financial data with mixed categorical and continuous features.
- **Native missing-value handling.** Several rolling features (e.g. `rolling_vol`, `spread_zscore`) are undefined at the start of each day or for new clients; the histogram variant routes missing values without imputation.
- **Robustness to feature scale.** Tree-based splits are invariant to monotone transformations, removing the need to standardise continuous features.
- **Speed.** The histogram approximation reduces per-iteration cost from $O(n \log n)$ to $O(n)$, making repeated cross-validation feasible on the full stacked dataset of $\approx$ 180,000 $\times$ 6 $\approx$ 1,080,000 rows.

Isotonic calibration. Raw HGBT probability scores are well-ranked but not necessarily well-calibrated. An `IsotonicRegression` is fitted on a held-out calibration fold (days 21–25 within the training period) to map raw scores to true probabilities. Isotonic regression is preferred over Platt scaling because it makes no parametric assumption about the shape of the calibration curve — appropriate given the non-Gaussian distribution of adversity scores across clients.

Data split. A strictly *chronological* 60/20/20 split is used: days 1–25 (train, 107,360 trades), days 26–33 (validation, 34,697 trades), days 34–42 (test, 38,843 trades). No shuffling is applied; this prevents any lookahead bias from future price information leaking into training labels.

Split	Accuracy	Precision	Recall	Log-loss
Train	0.5759	0.6053	0.2347	0.6764
Validation	0.5551	0.5233	0.2184	0.6860
Test	0.5564	0.5206	0.2160	0.6858

Table 1: Model performance averaged across all six horizons $\tau \in \{5, 10, 15, 20, 25, 30\}$. The base-rate entropy floor (adversity rate $\approx$ 0.46) is $H \approx 0.690$ nats; the model achieves log-loss 0.676–0.686 across splits, roughly 0.004–0.014 nats below the no-information baseline — near the practical signal ceiling for short-horizon adversity prediction from trade-level features.

The small train-to-test gap (accuracy 57.6% vs. 55.6%, log-loss 0.676 vs. 0.686) confirms that the model generalises without significant overfitting. The moderate recall ($\approx$ 0.22) reflects the deliberate choice to optimise log-loss (calibration) rather than a hard classification threshold.

2. Feature Selection and Justification

The feature vector has 16 dimensions: 15 trade-level features plus the horizon τ. All features are computed from information available *strictly before* the trade is executed, ensuring no lookahead.

#	Feature	Justification
1	`client_id`	Each client has a distinct adversity baseline (Task 1); encoding identity allows the model to learn per-client priors.
2	`side`	Buy (+1) and sell (−1) trades may exhibit asymmetric adversity due to order-book imbalances.
3	`volume`	Large trades are more likely to be information-driven; informed traders typically trade larger sizes.
4	`log_volume`	Log-transform compresses the heavy right tail of the volume distribution and captures relative size effects.
5	`spread`	A wider spread indicates higher uncertainty and lower liquidity; adverse flow concentrates when spreads are tight (adverse traders exploit narrow quotes).
6	`hour`	Adversity varies intraday; informed flow is often concentrated at the open and close.
7	`minute_of_day`	Finer-grained intraday signal than hour alone; captures lunch-time liquidity dips, etc.
8	`day_of_week`	Weekly seasonality in client flow patterns.
9	`eta`	Session progress $\in [0,1]$; end-of-day trades may carry larger directional bias as clients flatten positions.
10	`rolling_vol`	20-trade rolling RMS of mid-price returns; the primary volatility signal. High σ increases adversity risk across all clients.
11	`vol_spread`	Interaction term (volume $\times$ spread); large trades in wide markets are especially likely to move the mid against the LP.
12	`rolling_adv`	50-trade rolling mean of past adverse-trade indicators per client (lagged by 1); captures momentum in a client's adversity behaviour.
13	`order_flow`	Cumulative signed volume for the client within the day (excluding the current trade); a proxy for the client's directional pressure.
14	`spread_zscore`	$(s-\mu_{50})/\sigma_{50}$ per client; flags anomalously wide or narrow spreads relative to recent history.
15	`mom_short`	$(M_0 - M_0^{-5})/M_0^{-5}$: 5-trade price momentum; trades in the direction of a recent trend are more likely to be adverse for the LP.
16	τ	Horizon in seconds; included as a continuous feature so the single model can simultaneously learn the adversity-vs-τ relationship observed empirically in Task 1.

Table 2: Feature vector (16 dimensions). Features 1–15 are trade-level observables; feature 16 is the target horizon τ. All rolling quantities use per-client, within-day windows with a lag of 1 to prevent lookahead.

Feature ordering note. When calling `predict_adversity`, the feature vector must be supplied in the exact order above (indices 0–15), with τ as the final column. Rolling features default to `NaN` for the first N trades of each client-day; the model handles these natively via its internal missing-value splits.